Linux & Shell Lab 2: Find, Filter and Protect

Student worksheet | Name: _____ Group: _____ Date: _____

In Lab 1 you learned to move around and manage files. Today you will learn to **select many files at once, search inside files, combine commands, and control who can use a file**. You will finish by writing your first shell script. Type every command yourself and compare your result with the green boxes.

What you will learn

- Select groups of files with wildcards (`*`, `?`, `[ ]`)
- Search inside files with `grep` and find files with `find`
- Chain commands with pipes using `sort`, `uniq`, `cut` and `wc`
- Read and change permissions with `ls -l` and `chmod`
- Write and run a first shell script

Part 0: Recap and setup (10 min)

Quick recap. Without looking at your notes, write the command for each job:

1. Show the folder you are in: _____
2. Create a folder called `lab2`: _____
3. Write the word hi into a new file `a.txt`: _____
4. Add the word bye at the end of `a.txt`: _____

Create the practice folder and enter it:

```
cd ~
mkdir lab2
cd lab2
pwd
```

Expected output:

```
/home/student/lab2
```

Create the practice files. `printf` writes text, and `\n` means "start a new line". Type these exactly:

```
touch report1.txt report2.txt report3.txt notes.txt photo1.jpg photo2.jpg photo.png
printf "apple\nbanana\ncherry\napple\ndate\nbanana\napple\nelderberry\n" > fruits.txt
printf "INFO server started\nERROR disk full\nINFO user login\n" > log.txt
printf "WARNING low memory\nERROR network down\n" >> log.txt
printf "INFO user logout\nERROR disk full\n" >> log.txt
```

Check with `ls`, then look inside the two data files:

```
ls
cat -n fruits.txt
```

Expected output:

```
1■apple
2■banana
3■cherry
4■apple
5■date
6■banana
7■apple
8■elderberry
```

Part 1: Wildcards (15 min)

A **wildcard** is a special character that stands for other characters. The shell expands it into a list of matching file names **before** the command runs.

Wildcard	Meaning	Example
`*`	Any number of any characters (even none)	`ls *.txt`
`?`	Exactly one character	`ls report?.txt`
`[ ]`	One character from the list	`ls report[12].txt`

Try it (you are in `~/lab2`):

```
ls *.txt
```

Expected output:

```
fruits.txt  log.txt  notes.txt  report1.txt  report2.txt  report3.txt
```

```
ls report?.txt
```

Expected output:

```
report1.txt  report2.txt  report3.txt
```

```
ls photo*
```

Expected output:

```
photo.png  photo1.jpg  photo2.jpg
```

```
ls report[12].txt
```

Expected output:

```
report1.txt  report2.txt
```

Questions:

1. Which command lists only the JPEG pictures? _____
2. Why does `ls photo?.jpg` show `photo1.jpg` but not `photo.png`?
   _____
3. Which command lists `report1.txt` and `report3.txt` but not `report2.txt`?
   _____

Warning: wildcards work with every command, including `rm`. `rm *.txt` deletes **all** text files. Run `ls *.txt` first to see what would be affected.

Try it: copy all the report files into a new folder `backup` using one command.

```
mkdir backup
cp report*.txt backup
ls backup
```

Expected output:

```
report1.txt  report2.txt  report3.txt
```

Part 2: Searching inside files with grep (20 min)

`grep` prints the lines of a file that contain a word or pattern. Its name comes from "global regular expression print", but you only need to remember: **grep = search for text**.

Command	What it does
`grep word file`	Show lines containing `word`
`grep -n word file`	Also show line numbers
`grep -c word file`	Only count the matching lines
`grep -i word file`	Ignore upper/lower case
`grep -v word file`	Show lines that do **not** contain `word`
`grep -r word folder`	Search in all files inside a folder

Try it:

```
grep apple fruits.txt
```

Expected output:

```
apple
apple
apple
```

```
grep -c apple fruits.txt
```

Expected output:

```
3
```

```
grep ERROR log.txt
```

Expected output:

```
ERROR disk full
ERROR network down
ERROR disk full
```

```
grep -n ERROR log.txt
```

Expected output:

```
2:ERROR disk full
5:ERROR network down
7:ERROR disk full
```

```
grep -v INFO log.txt
```

Expected output:

```
ERROR disk full
WARNING low memory
ERROR network down
ERROR disk full
```

```
grep "disk full" log.txt
```

Expected output:

```
ERROR disk full
ERROR disk full
```

```
grep -r ERROR .
```

Expected output:

```
./log.txt:ERROR disk full
./log.txt:ERROR network down
./log.txt:ERROR disk full
```

Tip: use quotes when your search text contains a space, as in `"disk full"`.

Questions:

1. How many lines in `log.txt` contain WARNING? _____
2. Write a command that shows the lines NOT containing ERROR:
   _____
3. What does `grep -i error log.txt` show that `grep error log.txt` does not?
   _____

Part 3: Finding files with find (10 min)

`grep` searches **inside** files. `find` searches for **files and folders** by name or type.

```
find where -name "pattern"
find where -type d
```

- `where` is the folder to search; `.` means "here"
- `-name` matches file names (put the pattern in quotes so `find` expands it, not the shell)
- `-type d` finds only directories, `-type f` only regular files

Try it. First create a file in a deeper folder so there is something hard to spot by eye:

```
mkdir -p backup/old
touch backup/old/report9.txt
find . -name "report*"
```

Expected output:

```
./report1.txt
./report2.txt
./report3.txt
./backup/report1.txt
./backup/report2.txt
./backup/report3.txt
./backup/old/report9.txt
```

```
find . -name "*.jpg"
```

Expected output:

```
./photo1.jpg
./photo2.jpg
```

```
find . -type d
```

Expected output:

```
.
./backup
./backup/old
```

Questions:

1. Which command finds every `.txt` file under `~/lab2`? _____
2. Which command lists only the folders inside `~/lab2`? _____

Part 4: Pipes and filters (20 min)

In Lab 1 you used `|` once. Now you will use it as a **factory line**: each command does one small job and passes its result to the next one.

Command	Job
`sort`	Sorts lines alphabetically (`-n` numerically, `-r` reversed)
`uniq`	Removes **neighbouring** duplicate lines (so sort first)
`uniq -c`	Same, but prefixes each line with how many times it appeared
`cut -d " " -f1`	Keeps field 1 of each line, using a space as the separator
`wc -l`	Counts lines

Step by step. Run each command and watch how the result changes:

```
sort fruits.txt
```

Expected output:

```
apple
apple
apple
banana
banana
cherry
date
elderberry
```

```
sort fruits.txt | uniq
```

Expected output:

```
apple
banana
cherry
date
elderberry
```

```
sort fruits.txt | uniq -c
```

Expected output:

```
      3 apple
      2 banana
      1 cherry
      1 date
      1 elderberry
```

```
sort fruits.txt | uniq -c | sort -nr
```

Expected output:

```
      3 apple
      2 banana
      1 elderberry
      1 date
      1 cherry
```

Analyse the log. Count how many messages there are of each level (INFO, ERROR, WARNING):

```
cut -d " " -f1 log.txt | sort | uniq -c
```

Expected output:

```
      3 ERROR
      3 INFO
      1 WARNING
```

```
grep ERROR log.txt | wc -l
```

Expected output:

```
3
```

```
grep ERROR log.txt | sort | uniq -c
```

Expected output:

```
      2 ERROR disk full
      1 ERROR network down
```

Questions:

1. How many different fruits are in `fruits.txt`? _____
2. Which fruit appears most often, and how many times? _____
3. Which error message occurs twice? _____

Part 5: Permissions and your first script (30 min)

Every file has an **owner** and a set of **permissions** that say who may read, write or run it. Look at a detailed listing:

```
ls -l notes.txt
```

Expected output:

```
-rw-r--r-- 1 student student 0 Oct 17 11:31 notes.txt
```

The first block of ten characters is the key. Read it in four parts:

```
-   rw-   r--   r--
|   |     |     |
|   |     |     `-- others (everyone else)
|   |     `-------- group
|   `-------------- owner (user)
`------------------ type: - file, d directory
```

Letter	Meaning on a file	Number
`r`	read: see the content	4
`w`	write: change the content	2
`x`	execute: run it as a program	1
`-`	permission not given	0

So `rw-` = 4 + 2 + 0 = **6**, `r--` = **4**, and `rwx` = 4 + 2 + 1 = **7**. The file above is `644`: the owner can read and write; everyone else can only read.

Changing permissions with `chmod`: use numbers (`chmod 600 file`) or letters (`u`=owner, `g`=group, `o`=others, `+` add, `-` remove).

```
touch secret.txt
chmod 600 secret.txt
ls -l secret.txt
```

Expected output:

```
-rw------- 1 student student 0 Oct 17 11:31 secret.txt
```

```
chmod 644 secret.txt
ls -l secret.txt
```

Expected output:

```
-rw-r--r-- 1 student student 0 Oct 17 11:31 secret.txt
```

Questions:

1. What does `-rw-r-----` allow the group to do? _____
2. Which number means `rwxr-xr-x`? _____
3. Write the `chmod` number for: owner read+write, group nothing, others nothing.
   _____

Your first script. A shell script is a text file of commands. The first line `#!/bin/bash` tells Linux which shell to use. Use **single quotes** below, because in double quotes `!` can trigger an error in the shell.

```
echo '#!/bin/bash' > hello.sh
echo 'echo Hello from my first script' >> hello.sh
cat hello.sh
```

Expected output:

```
#!/bin/bash
echo Hello from my first script
```

Now try to run it. The `./` means "the file in this folder":

```
./hello.sh
```

Expected output:

```
bash: ./hello.sh: Permission denied
```

It fails because the file has no **execute** permission. Add it and run again:

```
ls -l hello.sh
chmod +x hello.sh
ls -l hello.sh
./hello.sh
```

Expected output:

```
-rw-r--r-- 1 student student 44 Oct 17 11:31 hello.sh
-rwxr-xr-x 1 student student 44 Oct 17 11:31 hello.sh
Hello from my first script
```

Questions:

4. What changed in the permission string after `chmod +x`? _____
5. Which command makes `hello.sh` readable and writable only by you, but still runnable by you?
   _____

Part 6: Mini challenge (15 min)

Work alone or in pairs. Use `man`, `--help` and your notes. Do not look at earlier parts.

1. In `~/lab2`, create a file `scores.txt` containing these five lines, one per line: `ali 12`, `sara 15`, `ali 18`, `sara 9`, `ali 14` (hint: `printf`).
2. Show only the lines of **ali** and count them.
3. Show the names only (first field), sorted, with a count for each name.
4. Create a script `count.sh` that prints the number of lines of `scores.txt`, using `wc -l scores.txt`. Make it executable and run it.
5. Run `find ~/lab2 -name "*.sh"` and check that your script appears.

When you finish, show your screen to your teacher. **Bonus:** change `count.sh` so it prints how many lines contain `ali`.

Cheat sheet

Goal	Command
All files ending in .txt	`ls *.txt`
Files with one-character number	`ls report?.txt`
Search text in a file / count / ignore case	`grep word file` / `grep -c word file` / `grep -i word file`
Lines NOT containing a word / with numbers	`grep -v word file` / `grep -n word file`
Search inside all files of a folder	`grep -r word folder`
Find files by name / find folders	`find . -name "*.txt"` / `find . -type d`
Sort / remove duplicates / count duplicates	`sort file` / `sort file | uniq` / `sort file | uniq -c`
Keep first word of each line	`cut -d " " -f1 file`
Count lines of command output	`command | wc -l`
Show permissions	`ls -l file`
Set permissions (numbers)	`chmod 644 file` (rw-r--r--) / `chmod 755 file` (rwxr-xr-x)
Make a file executable	`chmod +x file`
Run a script in this folder	`./script.sh`

Permission numbers: r = 4, w = 2, x = 1. Add them for each of owner, group and others.

Homework: Server Detective

Goal: practice wildcards, `grep`, pipes and permissions on your own, in about 45 minutes. Use the terminal only.

Part A: Build the data

1. Create a folder `~/detective`.
2. Inside it, create a file `access.log` with **at least 10 lines**. Each line has three words: a user name, an action (`login`, `logout` or `download`) and a file name or `none`. Example line: `ali download report.pdf`. Use at least 3 different users, and make sure some users appear more than once. (Hint: `printf`, or `echo` with `>` and `>>`.)
3. Create five empty files: `a1.log`, `a2.log`, `a3.log`, `b1.log`, `b2.log`.

Part B: Investigate with commands

4. List only `a1.log`, `a2.log` and `a3.log` (not `access.log`) using one wildcard.
5. Show all lines of `access.log` that contain the word `download`, and count them.
6. Show all lines of **one user** of your choice.
7. Show how many times **each user** appears, sorted from most to least active (hint: `cut`, `sort`, `uniq -c`, `sort -nr`).
8. Show how many lines do **not** contain `login`.
9. Find every `.log` file inside `~/detective` with `find`.

Part C: Permissions and script

10. Make `access.log` readable and writable only by you (`600`). Show the proof with `ls -l`.
11. Write a script `report.sh` that prints the number of lines of `access.log`. Make it executable and run it.
12. Run `ls -l` on the folder and copy the line for `report.sh` into your answers.

Part D: Explain in your own words (short written answers)

13. What is the difference between `*` and `?` in a wildcard?
14. Why do you need to run `sort` before `uniq`?
15. What does the permission string `-rwxr-x---` allow for the owner, the group and others?

Bonus (optional): write a one-line command that prints only the single line of the most active user in `access.log` (the count and the name). (Hint: add `head -n 1` to the end of the command from question 7.)

What to submit (one text file). From your home directory, run:

```
cd ~/detective
ls -l > ~/submission2.txt
history | tail -n 50 >> ~/submission2.txt
```

Open `~/submission2.txt` with `nano`, add your written answers for Part D at the bottom, and send it to your teacher by the deadline. **Deadline:** _____